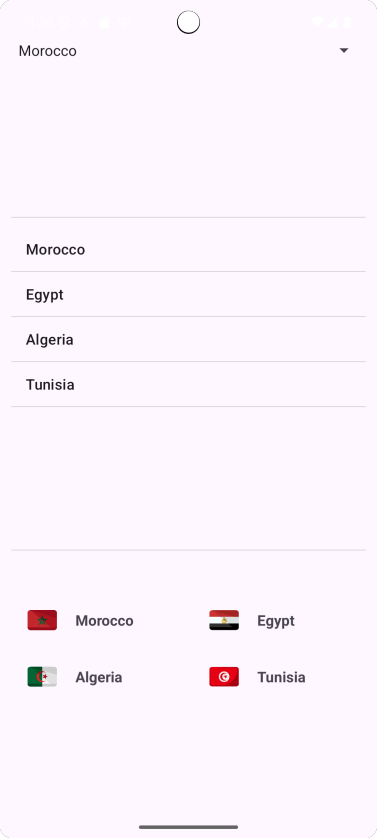
TP 5 : Vues à adaptateur en Java (ArrayAdapter)

Développement d'applications mobiles

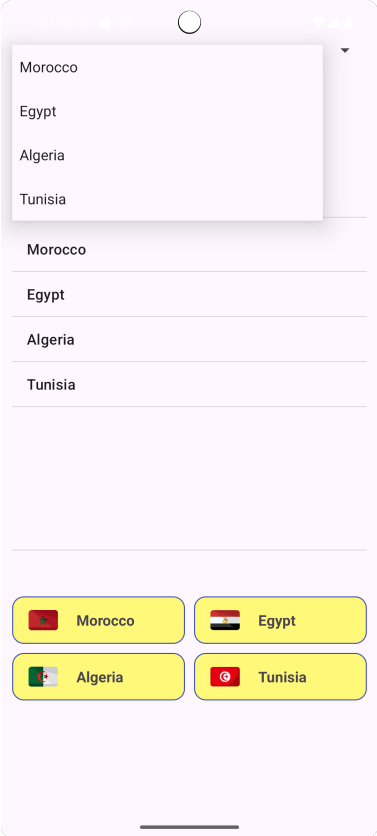
Objectifs pédagogiques

À la fin de ce TP, vous serez capable de :

- comprendre le rôle d'un `ArrayAdapter`
- utiliser un `ArrayAdapter` avec différentes vues :
  - `Spinner`
  - `ListView`
  - `GridView`
- créer un adaptateur personnalisé
- comprendre le rôle de `position` et de la méthode `getView()`



(a) Interface avec différentes vues (Spinner, ListView et GridView)



(b) Écran de sélection avec CheckBox

FIGURE 1 – Interface avec GridView en mode cartes

Consignes générales

- Après la partie démonstration , vous devez **appliquer les concepts du TP à un autre domaine de votre choix**, par exemple :
  - menu de café (boissons, desserts),
  - application de restaurant,
  - catalogue de produits,
  - application de bibliothèque ou de films.
- Chaque interface doit contenir **un seul type de vue à adaptateur** :
  - une interface avec un `Spinner`,
  - une interface avec un `ListView`,
  - une interface avec un `GridView`.
- La sélection effectuée dans une interface doit être **transmise à l'interface suivante** (à l'aide d'un `Intent`).
- Vous devez fournir une capture d'écran pour chaque vue utilisée :
  - interface Spinner,
  - interface ListView,
  - interface GridView,
  - interface finale affichant l'élément sélectionné.
- Les images et icônes doivent être téléchargées à partir de sites libres de droits tels que :
  - `https://www.flaticon.com`
  - `https://www.icons8.com`
- Une **fenêtre de connexion (Login)** est recommandée en tant qu'écran d'entrée de l'application.
- Les captures d'écran doivent être nommées clairement selon le format suivant : `tp5-ex1-spinner.p` `tp5-ex2-listview.png`, `tp5-ex3-gridview.png`, etc.
- Tout le travail doit être regroupé dans un **rapport nommé** : `TP5_AdapterView_NomPrenom.pdf`.
- Le rapport ainsi que le code source de l'application doivent être déposés dans **votre compte GitHub personnel**.

Contexte

On souhaite afficher une liste de pays en utilisant différentes vues Android. Chaque pays est caractérisé par :

- un nom
- un drapeau

La même collection de données sera utilisée pour toutes les vues.

Partie 1 : Création de l'entité `Country`

Créer la classe `Country` suivante :

```
public class Country {
    int flagResId;
    String name;

    public Country(int flagResId, String name) {
        this.flagResId = flagResId;
        this.name = name;
    }
```

```
}
```

—

Partie 2 : Initialisation des données avec `ArrayList`

Dans l'activité principale, créer une liste de pays en utilisant `ArrayList` :

```
Spinner spinner = findViewById(R.id.spinner);
ListView listView = findViewById(R.id.listView);
GridView gridView = findViewById(R.id.gridView);

ArrayList<Country> countries = new ArrayList<>();

countries.add(new Country(R.drawable.flag_sweden, "Sweden"));
countries.add(new Country(R.drawable.flag_switzerland, "Switzerland")
   );
countries.add(new Country(R.drawable.flag_taiwan, "Taiwan"));
countries.add(new Country(R.drawable.flag_france, "France"));
```

—

Partie 3 : Spinner avec `ArrayAdapter`

Créer un `Spinner` pour afficher uniquement les noms des pays.

```
ArrayList<String> countryNames = new ArrayList<>();

for (Country c : countries) {
    countryNames.add(c.name);
}

ArrayAdapter<String> spinnerAdapter =
    new ArrayAdapter<>(this,
        android.R.layout.simple_spinner_item,
        countryNames);

spinnerAdapter.setDropDownViewResource(
        android.R.layout.simple_spinner_dropdown_item);

spinner.setAdapter(spinnerAdapter);
```

Question : Quel est le rôle de `ArrayAdapter` dans ce cas ?

—

Partie 4 : ListView avec `ArrayAdapter` simple

Afficher les noms des pays dans un `ListView`.

```
ArrayAdapter<String> listAdapter =
    new ArrayAdapter<>(this,
        android.R.layout.simple_list_item_1,
        countryNames);

listView.setAdapter(listAdapter);
```

Question : Quelle est la différence entre l'affichage dans le `Spinner` et le `ListView` ?

—

Partie 5 : Création d'un adaptateur personnalisé

Pour afficher un drapeau et un nom dans chaque ligne, un adaptateur personnalisé est nécessaire.

Créer un layout `item_country.xml` contenant :

— un `ImageView` pour le drapeau

— un `TextView` pour le nom du pays

—

Partie 6 : Adaptateur personnalisé `CountryAdapter`

Compléter la classe suivante :

```
// Adaptateur personnalisé pour afficher des objets Country
public class CountryAdapter extends ArrayAdapter<Country> {

    // Référence vers l'activité qui utilise l'adaptateur
    Activity activity;

    // Identifiant du layout XML représentant un élément (ligne ou
       carte)
    int resourceId;

    // Liste des objets Country à afficher
    ArrayList<Country> items;

    // Constructeur de l'adaptateur
    public CountryAdapter(Activity activity,
                          int resourceId,
                          ArrayList<Country> items) {

        // Appel du constructeur de la classe ArrayAdapter
        super(activity, resourceId, items);

        // Initialisation des attributs
        this.activity = activity;
        this.resourceId = resourceId;
        this.items = items;
    }

    // Méthode appelée automatiquement pour afficher chaque élément
    @Override
    public View getView(int position,
                        View convertView,
                        ViewGroup parent) {

        // Vue représentant un élément de la liste ou de la grille
        View view = convertView;
```

```
        // Si la vue n'existe pas encore, on l'instancie à partir du
           XML
        if (view == null) {
            LayoutInflater inflater = activity.getLayoutInflater();
            view = inflater.inflate(resourceId, parent, false);
        }

        // Récupération des composants graphiques depuis le layout
        ImageView flagIV = view.findViewById(R.id.flagIV);
        TextView nameTV = view.findViewById(R.id.nameTV);

        // Récupération de l'objet Country correspondant à la
           position
        Country country = items.get(position);

        // Affectation des données aux composants graphiques
        flagIV.setImageResource(country.flagResId);
        nameTV.setText(country.name);

        // Retourne la vue finale à afficher
        return view;
    }
}
```

Question : Que représente le paramètre `position` dans la méthode `getView()` ?

Partie 7 : Le fichier `item_country.xml`

Lorsqu'on utilise un `ArrayAdapter` simple (par exemple avec `android.R.layout.simple_list_item_` chaque élément de la liste est affiché uniquement sous forme de texte.

Cependant, dans le cas d'un `GridView` ou d'un `ListView` personnalisé, chaque élément peut contenir plusieurs composants graphiques (comme une image et un texte). Il est donc nécessaire de créer un layout XML spécifique représentant **un seul élément** de la liste ou de la grille.

Le fichier `item_country.xml` définit la structure graphique d'un pays (drapeau + nom). Ce layout sera réutilisé automatiquement par l'adaptateur pour chaque élément de la collection, grâce à la méthode `getView()`.

Layout `item_country.xml`

Le layout suivant décrit l'apparence d'un pays sous forme d'une ligne ou d'une carte, composée d'une image (drapeau) et d'un texte (nom du pays).

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/
   android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:padding="10dp"
    android:gravity="center_vertical"
    android:background="@drawable/bg_card">
```

```
    <ImageView
        android:id="@+id/flagIV"
        android:layout_width="48dp"
        android:layout_height="32dp"
        android:contentDescription="flag"
        android:scaleType="fitCenter"/>

    <TextView
        android:id="@+id/nameTV"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="12dp"
        android:textSize="16sp"
        android:textStyle="bold"/>
</LinearLayout>
```

Ce layout est utilisé par l'adaptateur personnalisé pour afficher chaque élément du `ListView` ou du `GridView`.

Personnalisation de l'affichage avec `bg_card.xml`

Afin d'améliorer l'aspect visuel du `GridView`, un fond personnalisé peut être appliqué à chaque élément sous forme de carte. Ce fond est défini dans un fichier drawable XML nommé `bg_card.xml`.

L'utilisation d'un drawable XML permet de créer des formes légères, réutilisables et facilement personnalisables (couleur, coins arrondis, bordures), sans utiliser d'images.

Drawable `bg_card.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">

    <!-- Couleur de fond -->
    <solid android:color="#FFFFFF"/>

    <!-- Coins arrondis -->
    <corners android:radius="12dp"/>

    <!-- Bordure légère -->
    <stroke
        android:width="1dp"
        android:color="#DDDDDD"/>

    <!-- Espacement interne -->
    <padding
        android:left="8dp"
        android:top="8dp"
        android:right="8dp"
        android:bottom="8dp"/>
</shape>
```

Ce drawable est appliqué aux éléments du `GridView` via l'attribut : `android:background="@drawable/bg_card"` dans le layout `item_country.xml`.

Partie 8 : Utilisation avec ListView

Associer l'adaptateur personnalisé au `ListView` si vous voulez une liste avec icones :

```
CountryAdapter adapter =
    new CountryAdapter(this,
        R.layout.item_country,
        countries);

listView.setAdapter(adapter);
```

—

Partie 8 : Utilisation avec GridView

Réutiliser le **même adaptateur** avec un `GridView`.

```
gridView.setAdapter(adapter);
```

Question : Pourquoi peut-on utiliser le même adaptateur pour plusieurs vues ?

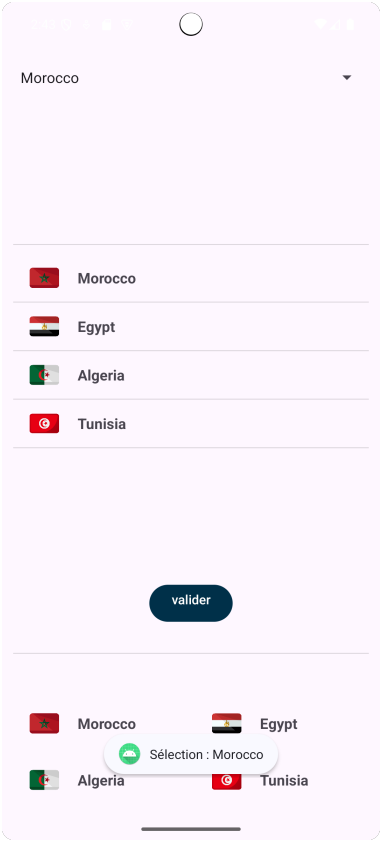


FIGURE 2 – Ajout des listener

Partie 9 : Gestion des sélections (Listeners)

Dans Android, chaque vue utilise un listener adapté à son comportement.

1. Spinner — `OnItemSelectedListener`

Le **Spinner** déclenche une action lorsqu'un élément est **sélectionné** (même sans clic explicite). On utilise donc `OnItemSelectedListener`.

— `onItemSelected()` : appelée quand l'utilisateur choisit un élément
— `onNothingSelected()` : appelée si aucun élément n'est sélectionné

Ce listener est adapté aux listes déroulantes.

2. ListView et GridView — `OnItemClickListener`

La **ListView** et la **GridView** réagissent à un **clic direct** sur un élément. On utilise donc `OnItemClickListener`.

— `position` permet d'identifier l'élément cliqué
— Les données sont récupérées via l'adaptateur

Ce listener est utilisé pour les listes et grilles interactives.

Code Java : Gestion des sélections avec les adaptateurs

```
// Bouton de validation
Button btnShow = findViewById(R.id.btnShow);

// Activer la sélection simple dans la ListView
listView.setChoiceMode(ListView.CHOICE_MODE_SINGLE);

// Clic sur le bouton : récupérer  l  élément sélectionné dans la
   ListView
btnShow.setOnClickListener(v -> {
    int position = listView.getCheckedItemPosition();

    if (position != ListView.INVALID_POSITION) {
        String selectedItem =
                countryAdapter.getItem(position).name;

        Toast.makeText(this,
                "Sélection␣:␣" + selectedItem,
                Toast.LENGTH_SHORT).show();
    } else {
        Toast.makeText(this,
                "Aucun␣élément␣sélectionné",
                Toast.LENGTH_SHORT).show();
    }
});

// Sélection  d un  élément dans le Spinner
spinner.setOnItemSelectedListener(
        new AdapterView.OnItemSelectedListener() {

    @Override
    public void onItemSelected(AdapterView<?> parent,
                               View view,
                               int position,
                               long id) {
```

```
        String selectedItem =
                spinnerAdapter.getItem(position);

        Toast.makeText(MainActivity2.this,
                "Sélection␣:␣" + selectedItem,
                Toast.LENGTH_SHORT).show();
    }

    @Override
    public void onNothingSelected(AdapterView<?> parent) { }
});

// Clic sur un élément du GridView
gridView.setOnItemClickListener(
        (parent, view, position, id) -> {

    Country country =
            countryAdapter.getItem(position);

    Toast.makeText(this,
            country.name,
            Toast.LENGTH_SHORT).show();
});
```

3. Bouton — Validation de la sélection

Le bouton **Valider** permet de confirmer l'élément sélectionné dans la ListView en utilisant `CHOICE_MODE_SINGLE` et `getCheckedItemPosition()`.

Conclusion

Dans ce TP, nous avons montré que :

- un `ArrayAdapter` relie une collection Java à une vue graphique
- différentes vues (`Spinner`, `ListView`, `GridView`) peuvent utiliser le même adaptateur
- la méthode `getView()` permet de personnaliser l'affichage de chaque élément